

Deep Sets

Manzil Zaheer^{1,2}, Satwik Kottur¹, Siamak Ravanbakhsh¹,
Barnabás Póczos¹, Ruslan Salakhutdinov¹, Alexander J Smola^{1,2}

¹ Carnegie Mellon University ² Amazon Web Services
{manzilz,skottur,mravanba,bapoczos,rsalakhu,smola}@cs.cmu.edu

Abstract

We study the problem of designing models for machine learning tasks defined on *sets*. In contrast to traditional approach of operating on fixed dimensional vectors, we consider objective functions defined on sets that are invariant to permutations. Such problems are widespread, ranging from estimation of population statistics [1], to anomaly detection in piezometer data of embankment dams [2], to cosmology [3, 4]. Our main theorem characterizes the permutation invariant functions and provides a family of functions to which any permutation invariant objective function must belong. This family of functions has a special structure which enables us to design a deep network architecture that can operate on sets and which can be deployed on a variety of scenarios including both unsupervised and supervised learning tasks. We also derive the necessary and sufficient conditions for permutation equivariance in deep models. We demonstrate the applicability of our method on population statistic estimation, point cloud classification, set expansion, and outlier detection.

1 Introduction

A typical machine learning algorithm, like regression or classification, is designed for fixed dimensional data instances. Their extensions to handle the case when the inputs or outputs are permutation invariant sets rather than fixed dimensional vectors is not trivial and researchers have only recently started to investigate them [5–8]. In this paper, we present a generic framework to deal with the setting where input and possibly output instances in a machine learning task are sets.

Similar to fixed dimensional data instances, we can characterize two learning paradigms in case of sets. In supervised learning, we have an output label for a set that is invariant or equivariant to the permutation of set elements. Examples include tasks like estimation of population statistics [1], where applications range from giga-scale cosmology [3, 4] to nano-scale quantum chemistry [9].

Next, there can be the unsupervised setting, where the “set” structure needs to be learned, e.g. by leveraging the homophily/heterophily tendencies within sets. An example is the task of set expansion (a.k.a. audience expansion), where given a set of objects that are similar to each other (e.g. set of words {lion, tiger, leopard}), our goal is to find new objects from a large pool of candidates such that the selected new objects are similar to the query set (e.g. find words like jaguar or cheetah among all English words). This is a standard problem in similarity search and metric learning, and a typical application is to find new image tags given a small set of possible tags. Likewise, in the field of computational advertisement, given a set of high-value customers, the goal would be to find similar people. This is an important problem in many scientific applications, e.g. given a small set of interesting celestial objects, astrophysicists might want to find similar ones in large sky surveys.

Main contributions. In this paper, (i) we propose a fundamental architecture, *DeepSets*, to deal with sets as inputs and show that the properties of this architecture are both necessary and sufficient (Sec. 2). (ii) We extend this architecture to allow for conditioning on arbitrary objects, and (iii) based on this architecture we develop a *deep network* that can operate on sets with possibly different sizes (Sec. 3). We show that a simple parameter-sharing scheme enables a general treatment of sets within supervised and semi-supervised settings. (iv) Finally, we demonstrate the wide applicability of our framework through experiments on diverse problems (Sec. 4).

2 Permutation Invariance and Equivariance

2.1 Problem Definition

A function f transforms its domain $\mathcal{X}$ into its range $\mathcal{Y}$. Usually, the input domain is a vector space $\mathbb{R}^d$ and the output response range is either a discrete space, e.g. $\{0, 1\}$ in case of classification, or a continuous space $\mathbb{R}$ in case of regression. Now, if the input is a set $X = \{x_1, \dots, x_M\}$, $x_m \in \mathfrak{X}$, i.e., the input domain is the power set $\mathcal{X} = 2^{\mathfrak{X}}$, then we would like the response of the function to be “indifferent” to the ordering of the elements. In other words,

Property 1 A function $f : 2^{\mathfrak{X}} \rightarrow \mathcal{Y}$ acting on sets must be permutation **invariant** to the order of objects in the set, i.e. for any permutation $\pi : f(\{x_1, \dots, x_M\}) = f(\{x_{\pi(1)}, \dots, x_{\pi(M)}\})$.

In the supervised setting, given N examples of $X^{(1)}, \dots, X^{(N)}$ as well as their labels $y^{(1)}, \dots, y^{(N)}$, the task would be to classify/regress (with variable number of predictors) while being **permutation invariant w.r.t. predictors**. Under unsupervised setting, the task would be to assign high scores to valid sets and low scores to improbable sets. These scores can then be used for set expansion tasks, such as image tagging or audience expansion in field of computational advertisement. In **transductive setting**, each instance $x_m^{(n)}$ has an associated labeled $y_m^{(n)}$. Then, the objective would be instead to learn a permutation **equivariant** function $\mathbf{f} : \mathfrak{X}^M \rightarrow \mathcal{Y}^M$ that upon permutation of the input instances permutes the output labels, i.e. for any permutation π :

$$\mathbf{f}([x_{\pi(1)}, \dots, x_{\pi(M)}]) = [f_{\pi(1)}(\mathbf{x}), \dots, f_{\pi(M)}(\mathbf{x})] \quad (1)$$

2.2 Structure

We want to study the structure of functions on sets. Their study in total generality is extremely difficult, so we analyze case-by-case. We begin by analyzing the **invariant case when $\mathfrak{X}$ is a countable set and $\mathcal{Y} = \mathbb{R}$** , where the next theorem characterizes its structure.

Theorem 2 A function $f(X)$ operating on a set X having elements from a countable universe, is a valid set function, i.e., **invariant** to the permutation of instances in X , iff it can be **decomposed** in the form $\rho(\sum_{x \in X} \phi(x))$, for suitable transformations ϕ and ρ .

The extension to case **when $\mathfrak{X}$ is uncountable**, like $\mathfrak{X} = \mathbb{R}$, we could only prove that $f(X) = \rho(\sum_{x \in X} \phi(x))$ holds for sets of fixed size. The proofs and difficulties in handling the uncountable case, are discussed in Appendix A. However, we still conjecture that exact equality holds in general.

Next, we analyze the **equivariant** case when $\mathfrak{X} = \mathcal{Y} = \mathbb{R}$ and $\mathbf{f}$ is restricted to be a neural network layer. The standard neural network layer is represented as $\mathbf{f}_{\Theta}(\mathbf{x}) = \sigma(\Theta \mathbf{x})$ where $\Theta \in \mathbb{R}^{M \times M}$ is the **weight vector** and $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is a nonlinearity such as sigmoid function. The following lemma states the necessary and sufficient conditions for permutation-equivariance in this type of function.

Lemma 3 The function $\mathbf{f}_{\Theta} : \mathbb{R}^M \rightarrow \mathbb{R}^M$ defined above is permutation **equivariant** iff all the off-diagonal elements of Θ are tied together and all the diagonal elements are equal as well. That is,

$$\Theta = \lambda \mathbf{I} + \gamma (\mathbf{1}\mathbf{1}^T) \quad \lambda, \gamma \in \mathbb{R} \quad \mathbf{1} = [1, \dots, 1]^T \in \mathbb{R}^M \quad \mathbf{I} \in \mathbb{R}^{M \times M} \text{ is the identity matrix}$$

This result can be easily extended to higher dimensions, i.e., $\mathfrak{X} = \mathbb{R}^d$ when λ, γ can be matrices.

2.3 Related Results

The general form of Theorem 2 is closely related with important results in different domains. Here, we quickly review some of these connections.

de Finetti theorem. A related concept is that of an exchangeable model in Bayesian statistics, It is backed by deFinetti’s theorem which states that any exchangeable model can be factored as

$$p(X|\alpha, M_0) = \int d\theta \left[\prod_{m=1}^M p(x_m|\theta) \right] p(\theta|\alpha, M_0), \quad (2)$$

where θ is some latent feature and α, M_0 are the hyper-parameters of the prior. To see that this fits into our result, let us consider exponential families with conjugate priors, where we can analytically calculate the integral of (2). In this special case $p(x|\theta) = \exp(\langle \phi(x), \theta \rangle - g(\theta))$ and $p(\theta|\alpha, M_0) = \exp(\langle \theta, \alpha \rangle - M_0 g(\theta) - h(\alpha, M_0))$. Now if we marginalize out θ , we get a form which looks exactly like the one in Theorem 2

$$p(X|\alpha, M_0) = \exp \left(h \left(\alpha + \sum_m \phi(x_m), M_0 + M \right) - h(\alpha, M_0) \right). \quad (3)$$

Representer theorem and kernel machines. Support distribution machines use $f(p) = \sum_i \alpha_i y_i K(p_i, p) + b$ as the prediction function [8, 10], where p_i, p are distributions and $\alpha_i, b \in \mathbb{R}$. In practice, the p_i, p distributions are never given to us explicitly, usually only i.i.d. sample sets are available from these distributions, and therefore we need to estimate kernel $K(p, q)$ using these samples. A popular approach is to use $\hat{K}(p, q) = \frac{1}{MM'} \sum_{i,j} k(x_i, y_j)$, where k is another kernel operating on the samples $\{x_i\}_{i=1}^M \sim p$ and $\{y_j\}_{j=1}^{M'} \sim q$. Now, these prediction functions can be seen fitting into the structure of our Theorem.

Spectral methods. A consequence of the polynomial decomposition is that spectral methods [11] can be viewed as a special case of the mapping $\rho \circ \phi(X)$: in that case one can compute polynomials, usually only up to a relatively low degree (such as $k = 3$), to perform inference about statistical properties of the distribution. The statistics are exchangeable in the data, hence they could be represented by the above map.

3 Deep Sets

3.1 Architecture

Invariant model. The structure of permutation invariant functions in Theorem 2 hints at a general strategy for inference over sets of objects, which we call DeepSets. Replacing ϕ and ρ by universal approximators leaves matters unchanged, since, in particular, ϕ and ρ can be used to approximate arbitrary polynomials. Then, it remains to learn these approximators, yielding in the following model:

- Each instance x_m is transformed (possibly by several layers) into some representation $\phi(x_m)$.
- The representations $\phi(x_m)$ are added up and the output is processed using the ρ network in the same manner as in any deep network (e.g. fully connected layers, nonlinearities, etc.).
- Optionally: If we have additional meta-information z , then the above mentioned networks could be conditioned to obtain the conditioning mapping $\phi(x_m|z)$.

In other words, **the key is to add up all representations and then apply nonlinear transformations.**

Equivariant model. Our goal is to design neural network layers that are equivariant to the permutations of elements in the input $\mathbf{x}$. Based on Lemma 3, a neural network layer $\mathbf{f}_\Theta(\mathbf{x})$ is permutation equivariant if and only if all the off-diagonal elements of Θ are tied together and all the diagonal elements are equal as well, i.e., $\Theta = \lambda \mathbf{I} + \gamma (\mathbf{1}\mathbf{1}^\top)$ for $\lambda, \gamma \in \mathbb{R}$. This function is simply a non-linearity applied to a weighted combination of (i) its input $\mathbf{I}\mathbf{x}$ and; (ii) the sum of input values $(\mathbf{1}\mathbf{1}^\top)\mathbf{x}$. **Since summation does not depend on the permutation, the layer is permutation-equivariant.** We can further manipulate the operations and parameters in this layer to get other **variations**, e.g.:

$$\mathbf{f}(\mathbf{x}) \doteq \sigma(\lambda \mathbf{I}\mathbf{x} + \gamma \max\text{pool}(\mathbf{x})\mathbf{1}). \quad (4)$$

where the maxpooling operation over elements of the set (similar to sum) is commutative. In practice, this variation performs better in some applications. This may be due to the fact that for $\lambda = \gamma$, the input to the non-linearity is max-normalized. Since composition of permutation equivariant functions is also permutation equivariant, we can build DeepSets by stacking such layers.

3.2 Other Related Works

Several recent works study equivariance and invariance in deep networks w.r.t. general group of transformations [12–14]. For example, [15] construct deep permutation invariant features by pairwise coupling of features at the previous layer, where $f_{i,j}([x_i, x_j]) \doteq [|x_i - x_j|, x_i + x_j]$ is invariant to transposition of i and j . Pairwise interactions within sets have also been studied in [16, 17]. [18] approach unordered instances by finding “good” orderings.

The idea of pooling a function across set-members is not new. In [19], pooling was used binary classification task for causality on a set of samples. [20] use pooling across a panoramic projection of 3D object for classification, while [21] perform pooling across multiple views. [22] observe the invariance of the payoff matrix in normal form games to the permutation of its rows and columns (i.e. player actions) and leverage pooling to predict the player action. The need of permutation equivariance also arise in deep learning over sensor networks and multi-agent settings, where a special case of Lemma 3 has been used as the architecture [23].

In light of these related works, we would like to **emphasize our novel contributions**: (i) the universality result of Theorem 2 for permutation invariance that also relates DeepSets to other machine learning techniques, see Sec. 3; (ii) the permutation equivariant layer of (4), which, according to Lemma 3 identifies necessary and sufficient form of parameter-sharing in a standard neural layer and; (iii) novel application settings that we study next.

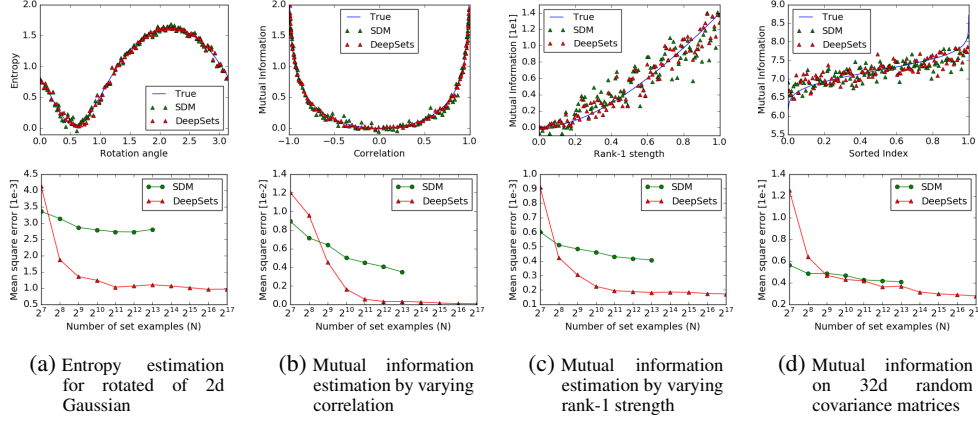


Figure 1: Population statistic estimation: Top set of figures, show prediction of DeepSets vs SDM for $N = 2^{10}$ case. Bottom set of figures, depict the mean squared error behavior as number of sets is increased. SDM has lower error for small N and DeepSets requires more data to reach similar accuracy. But for high dimensional problems DeepSets easily *scales* to large number of examples and produces much *lower* estimation error. Note that the $N \times N$ matrix inversion in SDM makes it prohibitively expensive for $N > 2^{14} = 16384$.

4 Applications and Empirical Results

We present a diverse set of applications for DeepSets. For the **supervised** setting, we apply DeepSets to **estimation of population statistics**, **sum of digits** and **classification of point-clouds**, and **regression with clustering side-information**. The **permutation-equivariant variation of DeepSets** is applied to the task of **outlier detection**. Finally, we investigate the application of DeepSets to **unsupervised set-expansion**, in particular, concept-set retrieval and image tagging. In most cases we compare our approach with the state-of-the art and report competitive results.

4.1 Set Input Scalar Response

4.1.1 Supervised Learning: Learning to Estimate Population Statistics

In the first experiment, we learn **entropy and mutual information of Gaussian distributions**, without providing any information about Gaussianity to DeepSets. The Gaussians are generated as follows:

- **Rotation:** We randomly chose a 2×2 covariance matrix Σ , and then generated N sample sets from $\mathcal{N}(0, R(\alpha)\Sigma R(\alpha)^T)$ of size $M = [300 - 500]$ for N random values of $\alpha \in [0, \pi]$. Our goal was to learn the entropy of the marginal distribution of first dimension. $R(\alpha)$ is the rotation matrix.
- **Correlation:** We randomly chose a $d \times d$ covariance matrix Σ for $d = 16$, and then generated N sample sets from $\mathcal{N}(0, [\Sigma, \alpha\Sigma; \alpha\Sigma, \Sigma])$ of size $M = [300 - 500]$ for N random values of $\alpha \in (-1, 1)$. Goal was to learn the mutual information of among the first d and last d dimension.
- **Rank 1:** We randomly chose $v \in \mathbb{R}^{32}$ and then generated a sample sets from $\mathcal{N}(0, I + \lambda vv^T)$ of size $M = [300 - 500]$ for N random values of $\lambda \in (0, 1)$. Goal was to learn the mutual information.
- **Random:** We chose N random $d \times d$ covariance matrices Σ for $d = 32$, and using each, generated a sample set from $\mathcal{N}(0, \Sigma)$ of size $M = [300 - 500]$. Goal was to learn the mutual information.

We train using L_2 loss with a DeepSets architecture having 3 fully connected layers with ReLU activation for both transformations ϕ and ρ . We compare against Support Distribution Machines (SDM) using a RBF kernel [10], and analyze the results in Fig. 1.

4.1.2 Sum of Digits

Next, we compare to what happens if our set data is treated as a sequence. We consider the task of finding sum of a given set of digits. We consider two variants of this experiment:

Text. We randomly sample a subset of maximum $M = 10$ digits from this dataset to build $100k$ “sets” of training images, where the set-label is sum of digits in that set. We test against sums of M digits, for M starting from 5 all the way up to 100 over another $100k$ examples.

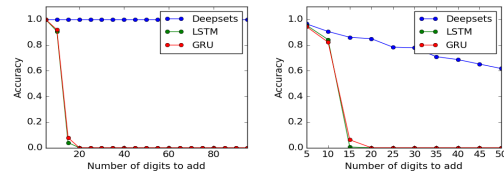


Figure 2: Accuracy of digit summation with text (left) and image (right) inputs. All approaches are trained on tasks of length 10 at most, tested on examples of length up to 100. We see that DeepSets generalizes better.

Image. MNIST8m [24] contains 8 million instances of 28×28 grey-scale stamps of digits in $\{0, \dots, 9\}$. We randomly sample a subset of maximum $M = 10$ images from this dataset to build $N = 100k$ “sets” of training and $100k$ sets of test images, where the **set-label is the sum of digits in that set (i.e. individual labels per image is unavailable)**. We test against sums of M images of MNIST digits, for M starting from 5 all the way up to 50.

We compare against recurrent neural networks – LSTM and GRU. All models are defined to have similar number of layers and parameters. The output of all models is a scalar, predicting the sum of N digits. Training is done on tasks of length 10 at most, while at test time we use examples of length up to 100. The accuracy, *i.e.* exact equality after rounding, is shown in Fig. 2. DeepSets generalize much better. Note for image case, the best classification error for single digit is around $p = 0.01$ for MNIST8m, so in a collection of N of images at least one image will be misclassified is $1 - (1 - p)^N$, which is 40% for $N = 50$. This matches closely with observed value in Fig. 2(b).

point-clouds simply matches closely with observed value in Fig. 2(b).

1. Deepsets treat each point independently by applying the element-wise transformation followed by aggregation

4.1.3 Point Cloud Classification

A point-cloud is a set of low-dimensional vectors. This type of data is frequently encountered in various applications like robotics, vision, and cosmology. In these applications, existing methods often convert the point-cloud data to voxel or mesh representation as a preprocessing step, *e.g.* [26, 29, 30]. Since the output of many range sensors, such as LiDAR, is in the form of point-cloud, direct application of deep learning methods to point-cloud is highly desirable. Moreover, it is easy and cheaper to apply transformations, such as rotation and translation, when working with point-clouds than voxelized 3D objects.

As **point-cloud data is just a set of points**, we can use DeepSets to classify point-cloud representation of a subset of ShapeNet objects [31], called ModelNet40 [25]. This subset consists of **3D representation of 9,843 training and 2,468 test instances belonging to 40 classes of objects**. We produce point-clouds with 100, 1000 and 5000 particles each (x, y, z -coordinates) from the mesh representation of objects using the point-cloud-library’s sampling routine [32]. Each set is normalized by the initial layer of the deep network to have zero mean (along individual axes) and unit (global) variance. Tab. 1 compares our method using three **permutation equivariant layers** against the competition; see Appendix H for details.

Model	Instance Size	Representation	Accuracy
3DShapeNets [25]	30^3	voxels (using convolutional deep belief net)	77%
VoxNet [26]	32^3	voxels (voxels from point-cloud + 3D CNN)	83.10%
MVCNN [21]	$164 \times 164 \times 12$	multi-view images (2D CNN + view-pooling)	90.1%
VRN Ensemble [27]	32^3	voxels (3D CNN, variational autoencoder)	95.54%
3D GAN [28]	64^3	voxels (3D CNN, generative adversarial training)	83.3%
DeepSets	5000×3	point-cloud	$90 \pm .3\%$
DeepSets	100×3	point-cloud	$82 \pm 2\%$

Table 1: Classification accuracy and the representation-size used by different methods on the ModelNet40.

4.1.4 Improved Red-shift Estimation Using Clustering Information

An important regression problem in cosmology is to estimate the red-shift of galaxies, corresponding to their age as well as their distance from us [33] based on **photometric observations**. One way to estimate the red-shift from photometric observations is using a regression model [34] on the galaxy clusters. The prediction for each galaxy does not change by permuting the members of the galaxy cluster. Therefore, we can **treat each galaxy cluster as a “set”** and use DeepSets to estimate the individual galaxy red-shifts. See Appendix G for more details.

For each galaxy, we have 17 photometric features from the redMaPPer galaxy cluster catalog [35] that contains photometric readings for 26,111 red galaxy clusters. Each galaxy-cluster in this catalog has between $\sim 20 - 300$ galaxies – *i.e.* $\mathbf{x} \in \mathbb{R}^{N(c) \times 17}$, where $N(c)$ is the cluster-size. The catalog also provides accurate spectroscopic red-shift estimates for a *subset* of these galaxies.

Method	Scatter
MLP	0.026
redMaPPer	0.025
DeepSets	0.023

Table 2: Red-shift experiment. Lower scatter is better.

We randomly split the data into 90% training and 10% test clusters, and minimize the squared loss of the prediction for available spectroscopic red-shifts. As it is customary in cosmology literature, we report the average **scatter** $\frac{|z_{\text{spec}} - z|}{1 + z_{\text{spec}}}$, where

z_{spec} is the accurate spectroscopic measurement and z is a photometric estimate in Tab. 2.

Method	LDA-1k (Vocab = 17k)					LDA-3k (Vocab = 38k)					LDA-5k (Vocab = 61k)				
	Recall (%)					Recall (%)					Recall (%)				
	@10	@100	@1k	MRR	Med.	@10	@100	@1k	MRR	Med.	@10	@100	@1k	MRR	Med.
Random	0.06	0.6	5.9	0.001	8520	0.02	0.2	2.6	0.000	28635	0.01	0.2	1.6	0.000	30600
Bayes Set	1.69	11.9	37.2	0.007	2848	2.01	14.5	36.5	0.008	3234	1.75	12.5	34.5	0.007	3590
w2v Near	6.00	28.1	54.7	0.021	641	4.80	21.2	43.2	0.016	2054	4.03	16.7	35.2	0.013	6900
NN-max	4.78	22.5	53.1	0.023	779	5.30	24.9	54.8	0.025	672	4.72	21.4	47.0	0.022	1320
NN-sum-con	4.58	19.8	48.5	0.021	1110	5.81	27.2	60.0	0.027	453	4.87	23.5	53.9	0.022	731
NN-max-con	3.36	16.9	46.6	0.018	1250	5.61	25.7	57.5	0.026	570	4.72	22.0	51.8	0.022	877
DeepSets	5.53	24.2	54.3	0.025	696	6.04	28.5	60.7	0.027	426	5.54	26.1	55.5	0.026	616

Table 3: Results on Text Concept Set Retrieval on LDA-1k, LDA-3k, and LDA-5k. Our DeepSets model outperforms other methods on LDA-3k and LDA-5k. However, all neural network based methods have inferior performance to w2v-Near baseline on LDA-1k, possibly due to small data size. Higher the better for recall@k and mean reciprocal rank (MRR). Lower the better for median rank (Med.)

4.2 Set Expansion unsupervised task (skipped)

In the set expansion task, we are given a set of objects that are similar to each other and our goal is to find new objects from a large pool of candidates such that the selected new objects are similar to the query set. To achieve this one needs to reason out the concept connecting the given set and then retrieve words based on their relevance to the inferred concept. It is an important task due to wide range of potential applications including personalized information retrieval, computational advertisement, tagging large amounts of unlabeled or weakly labeled datasets.

Going back to de Finetti’s theorem in Sec. 3.2, where we consider the marginal probability of a set of observations, the marginal probability allows for very simple metric for scoring additional elements to be added to X . In other words, this allows one to perform set expansion via the following score

$$s(x|X) = \log p(X \cup \{x\} | \alpha) - \log p(X | \alpha) p(\{x\} | \alpha) \quad (5)$$

Note that $s(x|X)$ is the point-wise mutual information between x and X . Moreover, due to exchangeability, it follows that regardless of the order of elements we have

$$S(X) = \sum_m s(x_m | \{x_{m-1}, \dots, x_1\}) = \log p(X | \alpha) - \sum_{m=1}^M \log p(\{x_m\} | \alpha) \quad (6)$$

When inferring sets, our goal is to find set completions $\{x_{m+1}, \dots, x_M\}$ for an initial set of query terms $\{x_1, \dots, x_m\}$, such that the aggregate set is coherent. This is the key idea of the Bayesian Set algorithm [36] (details in Appendix D). Using DeepSets, we can solve this problem in more generality as we can drop the assumption of data belonging to certain exponential family.

For learning the score $s(x|X)$, we take recourse to large-margin classification with structured loss functions [37] to obtain the relative loss objective $l(x, x'|X) = \max(0, s(x'|X) - s(x|X) + \Delta(x, x'))$. In other words, we want to ensure that $s(x|X) \geq s(x'|X) + \Delta(x, x')$ whenever x should be added and x' should not be added to X .

Conditioning. Often machine learning problems do not exist in isolation. For example, task like tag completion from a given set of tags is usually related to an object z , for example an image, that needs to be tagged. Such meta-data are usually abundant, e.g. author information in case of text, contextual data such as the user click history, or extra information collected with LiDAR point cloud.

Conditioning graphical models with meta-data is often complicated. For instance, in the Beta-Binomial model we need to ensure that the counts are always nonnegative, regardless of z . Fortunately, DeepSets does not suffer from such complications and the fusion of multiple sources of data can be done in a relatively straightforward manner. Any of the existing methods in deep learning, including feature concatenation by averaging, or by max-pooling, can be employed. Incorporating these meta-data often leads to significantly improved performance as will be shown in experiments; Sec. 4.2.2.

4.2.1 Text Concept Set Retrieval

In text concept set retrieval, the objective is to retrieve words belonging to a ‘concept’ or ‘cluster’, given few words from that particular concept. For example, given the set of words $\{tiger, lion, cheetah\}$, we would need to retrieve other related words like *jaguar, puma, etc*, which belong to the same concept of big cats. This task of concept set retrieval can be seen as a set completion task conditioned on the latent semantic concept, and therefore our DeepSets form a desirable approach.

Dataset. We construct a large dataset containing sets of $N_T = 50$ related words by extracting topics from latent Dirichlet allocation [38, 39], taken out-of-the-box¹. To compare across scales, we

¹github.com/dmlc/experimental-lda

consider three values of $k = \{1k, 3k, 5k\}$ giving us three datasets LDA-1k, LDA-3k, and LDA-5k, with corresponding vocabulary sizes of 17k, 38k, and 61k.

Methods. We learn this using a margin loss with a DeepSets architecture having 3 fully connected layers with ReLU activation for both transformations ϕ and ρ . Details of the architecture and training are in Appendix E. We compare to several baselines: (a) **Random** picks a word from the vocabulary uniformly at random. (b) **Bayes Set** [36]. (c) **w2v-Near** computes the nearest neighbors in the word2vec [40] space. Note that both Bayes Set and w2v NN are strong baselines. The former runs Bayesian inference using Beta-Binomial conjugate pair, while the latter uses the powerful 300 dimensional word2vec trained on the billion word GoogleNews corpus². (d) **NN-max** uses a similar architecture as our DeepSets but uses **max pooling** to compute the set feature, as opposed to sum pooling. (e) **NN-max-con** uses max pooling on set elements but concatenates this pooled representation with that of query for a final set feature. (f) **NN-sum-con** is similar to NN-max-con but uses sum pooling followed by concatenation with query representation.

Evaluation. We consider the standard retrieval metrics – recall@K, median rank and mean reciprocal rank, for evaluation. To elaborate, recall@K measures the number of true labels that were recovered in the top K retrieved words. We use three values of $K = \{10, 100, 1k\}$. The other two metrics, as the names suggest, are the median and mean of reciprocals of the true label ranks, respectively. Each dataset is split into TRAIN (80%), VAL (10%) and TEST (10%). We learn models using TRAIN and evaluate on TEST, while VAL is used for hyperparameter selection and early stopping.

Results and Observations. As seen in Tab. 3: (a) Our DeepSets model outperforms all other approaches on LDA-3k and LDA-5k by any metric, highlighting the significance of permutation invariance property. (b) On LDA-1k, our model does not perform well when compared to w2v-Near. We hypothesize that this is due to small size of the dataset insufficient to train a high capacity neural network, while w2v-Near has been trained on a billion word corpus. Nevertheless, our approach comes the closest to w2v-Near amongst other approaches, and is only 0.5% lower by Recall@10.

4.2.2 Image Tagging

We next experiment with image tagging, where the task is to retrieve all relevant tags corresponding to an image. Images usually have only a subset of relevant tags, therefore predicting other tags can help enrich information that can further be leveraged in a downstream supervised task. In our setup, we learn to predict tags by conditioning DeepSets on the image, *i.e.*, we train to predict a partial set of tags from the image and remaining tags. At test time, we predict tags from the image alone.

Datasets. We report results on the following three datasets - ESPGame, IAPRTC-12.5 and our in-house dataset, COCO-Tag. We refer the reader to Appendix F, for more details about datasets.

Methods. The setup for DeepSets to tag images is similar to that described in Sec. 4.2.1. The only difference being the conditioning on the image features, which is concatenated with the set feature obtained from pooling individual element representations.

Baselines. We perform comparisons against several baselines, previously reported in [41]. Specifically, we have Least Sq., a ridge regression model, MBRM [42], JEC [43] and FastTag [41]. Note that these methods do not use deep features for images, which could lead to an unfair comparison. As there is no publicly available code for MBRM and JEC, we cannot get performances of these models with Resnet extracted features. However, we report results with deep features for FastTag and Least Sq., using code made available by the authors³.

Evaluation. For ESPgame and IAPRTC-12.5, we follow the evaluation metrics as in [44]—precision (P), recall (R), F1 score (F1), and number of tags with non-zero recall (N+). These metrics are evaluate for each tag and the mean is reported (see [44] for further details). For COCO-Tag, however, we use recall@K for three values of $K = \{10, 100, 1000\}$, along with median rank and mean reciprocal rank (see evaluation in Sec. 4.2.1 for metric details).

Method	ESP game				IAPRTC-12.5			
	P	R	F1	N+	P	R	F1	N+
Least Sq.	35	19	25	215	40	19	26	198
MBRM	18	19	18	209	24	23	23	223
JEC	24	19	21	222	29	19	23	211
FastTag	46	22	30	247	47	26	34	280
Least Sq.(D)	44	32	37	232	46	30	36	218
FastTag(D)	44	32	37	229	46	33	38	254
DeepSets	39	34	36	246	42	31	36	247

Table 4: Results of image tagging on ESPgame and IAPRTC-12.5 datasets. Performance of our DeepSets approach is roughly similar to the best competing approaches, except for precision. Refer text for more details. Higher the better for all metrics – precision (P), recall (R), f1 score (F1), and number of non-zero recall tags (N+).

²code.google.com/archive/p/word2vec/

³<http://www.cse.wustl.edu/~mchen/>



Figure 3: Each row shows a set, constructed from CelebA dataset, such that all set members except for an outlier, share at least two attributes (on the right). The outlier is identified with a red frame. The model is trained by observing examples of sets and their anomalous members, **without access to the attributes**. The probability assigned to each member by the outlier detection network is visualized using a red bar at the bottom of each image. The probabilities in each row sum to one.

Results and Observations. Tab. 4 shows results of image tagging on ESPgame and IAPRTC-12.5, and Tab. 5 on COCO-Tag. Here are the key observations from Tab. 4: (a) performance of our DeepSets model is comparable to the best approaches on all metrics but precision, (b) our recall beats the best approach by 2% in ESPgame. On further investigation, we found that the DeepSets model retrieves more relevant tags, which are not present in list of ground truth tags due to a limited 5 tag annotation. Thus, this takes a toll on precision while gaining on recall, yet yielding improvement on F1. On the larger and richer COCO-Tag, we see that the DeepSets approach outperforms other methods comprehensively, as expected. Qualitative examples are in Appendix F.

Method	Recall			MRR	Med.
	@10	@100	@1k		
w2v NN (blind)	5.6	20.0	54.2	0.021	823
DeepSets (blind)	9.0	39.2	71.3	0.044	310
DeepSets	31.4	73.4	95.3	0.131	28

Table 5: Results on COCO-Tag dataset. Clearly, DeepSets outperforms other baselines significantly. Higher the better for recall@K and mean reciprocal rank (MRR). Lower the better for median rank (Med).

4.3 Set Anomaly Detection

The objective here is to find the anomalous face in each set, simply by observing examples and without any access to the attribute values. CelebA dataset [45] contains 202,599 face images, each annotated with 40 boolean attributes. We build $N = 18,000$ sets of 64×64 stamps, using these attributes each containing $M = 16$ images (on the training set) as follows: randomly select 2 attributes, draw 15 images having those attributes, and a single target image where both attributes are absent. Using a similar procedure we build sets on the test images. No individual person’s face appears in both train and test sets. Our deep neural network consists of 9 2D-convolution and max-pooling layers followed by 3 permutation-equivariant layers, and finally a softmax layer that assigns a probability value to each set member (Note that one could identify arbitrary number of outliers using a sigmoid activation at the output). Our trained model successfully finds the anomalous face in **75% of test sets**. Visually inspecting these instances suggests that the task is non-trivial even for humans; see Fig. 3.

As a *baseline*, we repeat the same experiment by using a set-pooling layer after convolution layers, and replacing the permutation-equivariant layers with fully connected layers of same size, where the final layer is a 16-way softmax. The resulting network shares the convolution filters for all instances within all sets, however the input to the softmax is not equivariant to the permutation of input images. Permutation equivariance seems to be crucial here as the baseline model achieves a training and **test accuracy of $\sim 6.3\%$** ; the same as random selection. See Appendix I for more details.

5 Summary

In this paper, we develop DeepSets, a model based on powerful **permutation invariance and equivariance properties**, along with the theory to support its performance. We demonstrate the generalization ability of DeepSets across several domains by extensive experiments, and show both qualitative and quantitative results. In particular, we explicitly show that DeepSets outperforms other intuitive deep networks, which are not backed by theory (Sec. 4.2.1, Sec. 4.1.2). Last but not least, it is worth noting that **the state-of-the-art we compare to is a specialized technique for each task, whereas our one model, i.e., DeepSets, is competitive across the board**.

References

- [1] B. Poczos, A. Rinaldo, A. Singh, and L. Wasserman. Distribution-free distribution regression. In *International Conference on AI and Statistics (AISTATS)*, JMLR Workshop and Conference Proceedings, 2013. pages 1
- [2] I. Jung, M. Berges, J. Garrett, and B. Poczos. Exploration and evaluation of ar, mpca and kl anomaly detection techniques to embankment dam piezometer data. *Advanced Engineering Informatics*, 2015. pages 1
- [3] M. Ntampaka, H. Trac, D. Sutherland, S. Fromenteau, B. Poczos, and J. Schneider. Dynamical mass measurements of contaminated galaxy clusters using machine learning. *The Astrophysical Journal*, 2016. URL <http://arxiv.org/abs/1509.05409>. pages 1
- [4] M. Ravanbakhsh, J. Oliva, S. Fromenteau, L. Price, S. Ho, J. Schneider, and B. Poczos. Estimating cosmological parameters from the dark matter distribution. In *International Conference on Machine Learning (ICML)*, 2016. pages 1
- [5] J. Oliva, B. Poczos, and J. Schneider. Distribution to distribution regression. In *International Conference on Machine Learning (ICML)*, 2013. pages 1
- [6] Z. Szabo, B. Sriperumbudur, B. Poczos, and A. Gretton. Learning theory for distribution regression. *Journal of Machine Learning Research*, 2016. pages
- [7] K. Muandet, D. Balduzzi, and B. Schoelkopf. Domain generalization via invariant feature representation. In *In Proceeding of the 30th International Conference on Machine Learning (ICML 2013)*, 2013. pages
- [8] K. Muandet, K. Fukumizu, F. Dinuzzo, and B. Schoelkopf. Learning from distributions via support measure machines. In *In Proceeding of the 26th Annual Conference on Neural Information Processing Systems (NIPS 2012)*, 2012. pages 1, 3
- [9] Felix A. Faber, Alexander Lindmaa, O. Anatole von Lilienfeld, and Rickard Armiento. Machine learning energies of 2 million elpasolite (abC_2D_6) crystals. *Phys. Rev. Lett.*, 117:135502, Sep 2016. doi: 10.1103/PhysRevLett.117.135502. pages 1
- [10] B. Poczos, L. Xiong, D. Sutherland, and J. Schneider. Support distribution machines, 2012. URL <http://arxiv.org/abs/1202.0302>. pages 3, 4
- [11] A. Anandkumar, R. Ge, D. Hsu, S. M. Kakade, and M. Telgarsky. Tensor decompositions for learning latent variable models. *arXiv preprint arXiv:1210.7559*, 2012. pages 3
- [12] Robert Gens and Pedro M Domingos. Deep symmetry networks. In *Advances in neural information processing systems*, pages 2537–2545, 2014. pages 3
- [13] Taco S Cohen and Max Welling. Group equivariant convolutional networks. *arXiv preprint arXiv:1602.07576*, 2016. pages
- [14] Siamak Ravanbakhsh, Jeff Schneider, and Barnabas Poczos. Equivariance through parameter-sharing. *arXiv preprint arXiv:1702.08389*, 2017. pages 3
- [15] Xu Chen, Xiuyuan Cheng, and Stéphane Mallat. Unsupervised deep haar scattering on graphs. In *Advances in Neural Information Processing Systems*, pages 1709–1717, 2014. pages 3
- [16] Michael B Chang, Tomer Ullman, Antonio Torralba, and Joshua B Tenenbaum. A compositional object-based approach to learning physical dynamics. *arXiv preprint arXiv:1612.00341*, 2016. pages 3
- [17] Nicholas Guttenberg, Nathaniel Virgo, Olaf Witkowski, Hidetoshi Aoki, and Ryota Kanai. Permutation-equivariant neural networks applied to dynamics prediction. *arXiv preprint arXiv:1612.04530*, 2016. pages 3
- [18] Oriol Vinyals, Samy Bengio, and Manjunath Kudlur. Order matters: Sequence to sequence for sets. *arXiv preprint arXiv:1511.06391*, 2015. pages 3
- [19] David Lopez-Paz, Robert Nishihara, Soumith Chintala, Bernhard Schölkopf, and Léon Bottou. Discovering causal signals in images. *arXiv preprint arXiv:1605.08179*, 2016. pages 3

- [20] Baoguang Shi, Song Bai, Zhichao Zhou, and Xiang Bai. Deeppano: Deep panoramic representation for 3-d shape recognition. *IEEE Signal Processing Letters*, 22(12):2339–2343, 2015. pages 3, 26, 27
- [21] Hang Su, Subhransu Maji, Evangelos Kalogerakis, and Erik Learned-Miller. Multi-view convolutional neural networks for 3d shape recognition. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 945–953, 2015. pages 3, 5, 26, 27
- [22] Jason S Hartford, James R Wright, and Kevin Leyton-Brown. Deep learning for predicting human strategic behavior. In *Advances in Neural Information Processing Systems*, pages 2424–2432, 2016. pages 3
- [23] Sainbayar Sukhbaatar, Rob Fergus, et al. Learning multiagent communication with backpropagation. In *Neural Information Processing Systems*, pages 2244–2252, 2016. pages 3
- [24] Gaëlle Loosli, Stéphane Canu, and Léon Bottou. Training invariant support vector machines using selective sampling. In Léon Bottou, Olivier Chapelle, Dennis DeCoste, and Jason Weston, editors, *Large Scale Kernel Machines*, pages 301–320. MIT Press, Cambridge, MA., 2007. pages 5
- [25] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 3d shapenets: A deep representation for volumetric shapes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1912–1920, 2015. pages 5, 26
- [26] Daniel Maturana and Sebastian Scherer. Voxnet: A 3d convolutional neural network for real-time object recognition. In *Intelligent Robots and Systems (IROS), 2015 IEEE/RSJ International Conference on*, pages 922–928. IEEE, 2015. pages 5, 26
- [27] Andrew Brock, Theodore Lim, JM Ritchie, and Nick Weston. Generative and discriminative voxel modeling with convolutional neural networks. *arXiv preprint arXiv:1608.04236*, 2016. pages 5, 26
- [28] Jiajun Wu, Chengkai Zhang, Tianfan Xue, William T Freeman, and Joshua B Tenenbaum. Learning a probabilistic latent space of object shapes via 3d generative-adversarial modeling. *arXiv preprint arXiv:1610.07584*, 2016. pages 5, 26
- [29] Siamak Ravanbakhsh, Junier Oliva, Sebastien Fromenteau, Layne C Price, Shirley Ho, Jeff Schneider, and Barnabás Póczos. Estimating cosmological parameters from the dark matter distribution. In *Proceedings of The 33rd International Conference on Machine Learning*, 2016. pages 5
- [30] Hong-Wei Lin, Chiew-Lan Tai, and Guo-Jin Wang. A mesh reconstruction algorithm driven by an intrinsic property of a point cloud. *Computer-Aided Design*, 36(1):1–9, 2004. pages 5
- [31] Angel X Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, et al. Shapenet: An information-rich 3d model repository. *arXiv preprint arXiv:1512.03012*, 2015. pages 5
- [32] Radu Bogdan Rusu and Steve Cousins. 3D is here: Point Cloud Library (PCL). In *IEEE International Conference on Robotics and Automation (ICRA)*, Shanghai, China, May 9-13 2011. pages 5
- [33] James Binney and Michael Merrifield. *Galactic astronomy*. Princeton University Press, 1998. pages 5, 25
- [34] AJ Connolly, I Csabai, AS Szalay, DC Koo, RG Kron, and JA Munn. Slicing through multicolor space: Galaxy redshifts from broadband photometry. *arXiv preprint astro-ph/9508100*, 1995. pages 5, 25
- [35] Eduardo Roza and Eli S Rykoff. redmapper ii: X-ray and sz performance benchmarks for the sdss catalog. *The Astrophysical Journal*, 783(2):80, 2014. pages 5, 25
- [36] Zoubin Ghahramani and Katherine A Heller. Bayesian sets. In *NIPS*, volume 2, pages 22–23, 2005. pages 6, 7, 20, 21, 22

- [37] B. Taskar, C. Guestrin, and D. Koller. Max-margin Markov networks. In S. Thrun, L. Saul, and B. Schölkopf, editors, *Advances in Neural Information Processing Systems 16*, pages 25–32, Cambridge, MA, 2004. MIT Press. pages 6
- [38] Jonathan K. Pritchard, Matthew Stephens, and Peter Donnelly. Inference of population structure using multilocus genotype data. *Genetics*, 155(2):945–959, 2000. ISSN 0016-6731. URL <http://www.genetics.org/content/155/2/945>. pages 6, 22
- [39] David M. Blei, Andrew Y. Ng, Michael I. Jordan, and John Lafferty. Latent dirichlet allocation. *Journal of Machine Learning Research*, 3:2003, 2003. pages 6, 22
- [40] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *Advances in neural information processing systems*, pages 3111–3119, 2013. pages 7, 22
- [41] Minmin Chen, Alice Zheng, and Kilian Weinberger. Fast image tagging. In *Proceedings of The 30th International Conference on Machine Learning*, pages 1274–1282, 2013. pages 7, 23
- [42] S. L. Feng, R. Manmatha, and V. Lavrenko. Multiple bernoulli relevance models for image and video annotation. In *Proceedings of the 2004 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, CVPR’04*, pages 1002–1009, Washington, DC, USA, 2004. IEEE Computer Society. pages 7, 23
- [43] Ameesh Makadia, Vladimir Pavlovic, and Sanjiv Kumar. A new baseline for image annotation. In *Proceedings of the 10th European Conference on Computer Vision: Part III, ECCV ’08*, pages 316–329, Berlin, Heidelberg, 2008. Springer-Verlag. pages 7, 23
- [44] Matthieu Guillaumin, Thomas Mensink, Jakob Verbeek, and Cordelia Schmid. Tagprop: Discriminative metric learning in nearest neighbor models for image auto-annotation. In *Computer Vision, 2009 IEEE 12th International Conference on*, pages 309–316. IEEE, 2009. pages 7, 23, 24
- [45] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Deep learning face attributes in the wild. In *Proceedings of International Conference on Computer Vision (ICCV)*, 2015. pages 8
- [46] Branko Ćurgus and Vania Mascioni. Roots and polynomials as homeomorphic spaces. *Expositiones Mathematicae*, 24(1):81–95, 2006. pages 13, 15
- [47] Boris A Khesin and Serge L Tabachnikov. *Arnold: Swimming Against the Tide*, volume 86. American Mathematical Society, 2014. pages 15
- [48] Jerrold E Marsden and Michael J Hoffman. *Elementary classical analysis*. Macmillan, 1993. pages 15
- [49] Nicolas Bourbaki. *Eléments de mathématiques: théorie des ensembles, chapitres 1 à 4*, volume 1. Masson, 1990. pages 15
- [50] C. A. Micchelli. Interpolation of scattered data: distance matrices and conditionally positive definite functions. *Constructive Approximation*, 2:11–22, 1986. pages 18
- [51] Luis Von Ahn and Laura Dabbish. Labeling images with a computer game. In *Proceedings of the SIGCHI conference on Human factors in computing systems*, pages 319–326. ACM, 2004. pages 23
- [52] Michael Grubinger. Analysis and evaluation of visual information systems performance, 2007. URL <http://eprints.vu.edu.au/1435>. Thesis (Ph. D.)—Victoria University (Melbourne, Vic.), 2007. pages 23
- [53] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European Conference on Computer Vision*, pages 740–755. Springer, 2014. pages 23
- [54] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. pages 25, 26, 27
- [55] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (elus). *arXiv preprint arXiv:1511.07289*, 2015. pages 27